PROJECT CASE STUDY: Advanced SQL Sales Data Warehouse

1. Project Executive Summary

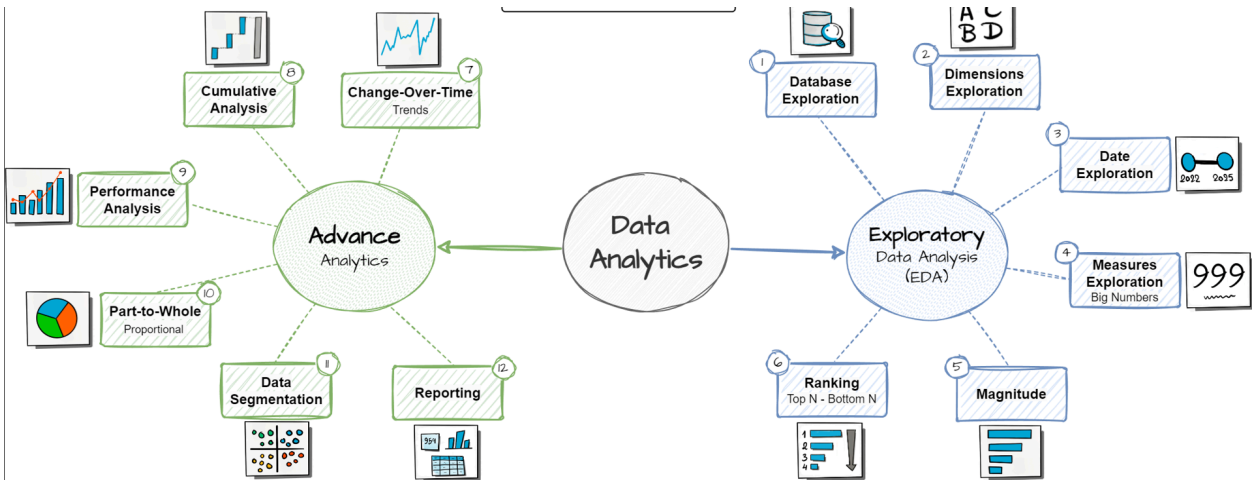
*This project demonstrates the end-to-end development of a scalable data infrastructure. The primary objective was to move away from flat-file data storage into a structured **Star Schema** environment, enabling complex analytical operations like customer lifecycle tracking, trend smoothing, and multi-tier data segmentation.*

---

2. Architectural Design: Star Schema Implementation

To ensure high query performance and logical data separation, the project utilizes a Star Schema architecture. This design separates quantitative business metrics from descriptive attributes, allowing for efficient "Slice and Dice" analysis.

- ***Central Fact Table (`fact_sales`):** Contains the core business measures: Sales Amount, Quantity, and Price.*
- ***Dimension Tables (`dim_customers`, `dim_products`):** Contains the descriptive metadata required for granular filtering and grouping (Geography, Demographics, and Categories).*



---

3. The Data Pipeline (ETL Logic)

*A significant portion of the project involved the **Transform** phase of ETL to ensure the warehouse was "Business-Ready":*

- ***Logic-Based Cleaning:*** *Standardizing customer identities and handling date-based calculations (Lifespan and Age) using system-level functions.*
- ***Feature Engineering:*** *Creating derived attributes within the database to support advanced KPIs without needing external processing.*

---

4. Technical Deep-Dive: Advanced SQL Analytics

This project utilizes high-level SQL techniques to extract deep insights that standard aggregations cannot provide.

```
case when lifespan >= 12 and total_sales > 5000 then 'VIP'
          when lifespan >= 12 and total_sales < 5000 then 'Regular'
      else 'New'
      end customer_segment,
```

A. Automated Reporting Views (`Report_Customers.sql`)

*I developed a modular **SQL View** that serves as a permanent analytical layer. It automates the calculation of:*

- ***Recency Analysis:*** *Calculating the gap between the current date and the last transaction to identify database freshness.*
- ***Monetary Distribution:*** *Computing Average Monthly Spend and Average Order Value (AOV) at the individual level.*

B. Tiered Data Segmentation

Instead of static lists, I implemented dynamic logic to categorize the database into three actionable segments:

- ***High-Value (VIP):*** *Defined by a specific intersection of longevity (12+ months) and high-tier spending.*
- ***Established (Regular):*** *Long-term data points with stable, baseline metrics.*
- ***Acquisition (New):*** *Entries with a short historical lifespan requiring different analytical tracking.*

C. Time-Series & Cumulative Analysis (`Analysis.sql`)

To understand growth velocity, I utilized ***Window Functions****:*

- ***Running Totals:*** *Implementing `SUM() OVER()` logic to visualize the accumulation of revenue over time.*
- ***Smoothing Trends:*** *Using moving averages to remove "noise" from seasonal data fluctuations.*

```sql
create view report_customers as
with base_query as (

/*---------------------------------------------------------------------------
1) Base Query: Retrieves core columns from fact_sales and dim_products
---------------------------------------------------------------------------*/

select f.order_number,f.product_key,f.order_date,f.sales_amount,f.quantity,c.customer_key,c.customer_number,
concat(c.first_name, ' ' , c.last_name) as customer_name ,
timestampdiff(year,c.birthdate,now()) as age
from fact_sales f join dim_customers c on c.customer_key = f.customer_key )

, customer_aggregation as (

/*---------------------------------------------------------------------------
2) Product Aggregations: Summarizes key metrics at the product level
---------------------------------------------------------------------------*/

select customer_key, customer_number,customer_name,age,count(distinct order_number) as total_orders,
sum(sales_amount) as total_sales,sum(quantity) as total_quantity,count(distinct product_key) as total_products,
max(order_date) as last_order_date,
timestampdiff(month,min(order_date),max(order_date)) as lifespan
from base_query
group by customer_key, customer_number,customer_name,age)

/*---------------------------------------------------------------------------
  3) Final Query: Combines all product results into one output
---------------------------------------------------------------------------*/

select customer_key, customer_number,customer_name,age,total_orders,total_sales,total_quantity,total_products,last_order_date,lifespan,timestampdiff(month,last_order_date,now()) as recency,

case when age < 20 then 'Under 20'
         when age between 20 and 29  then '20-29'
     when age between 30 and 39  then '30-39'
     when age between 40 and 49  then '40-49'
     else '50 and above'
end as age_group,

case when lifespan >= 12 and total_sales > 5000 then 'VIP'
          when lifespan >= 12 and total_sales < 5000 then 'Regular'
      else 'New'
      end customer_segment,

-- compute average order value
case when total_sales = 0 then 0
         else round(total_sales / total_orders,0)
end as avg_order_value,

-- compute average monthly spend

case when lifespan = 0 then total_sales
         else round(total_sales / lifespan ,0)
end as avg_monthly_spend

from customer_aggregation ;
```

---

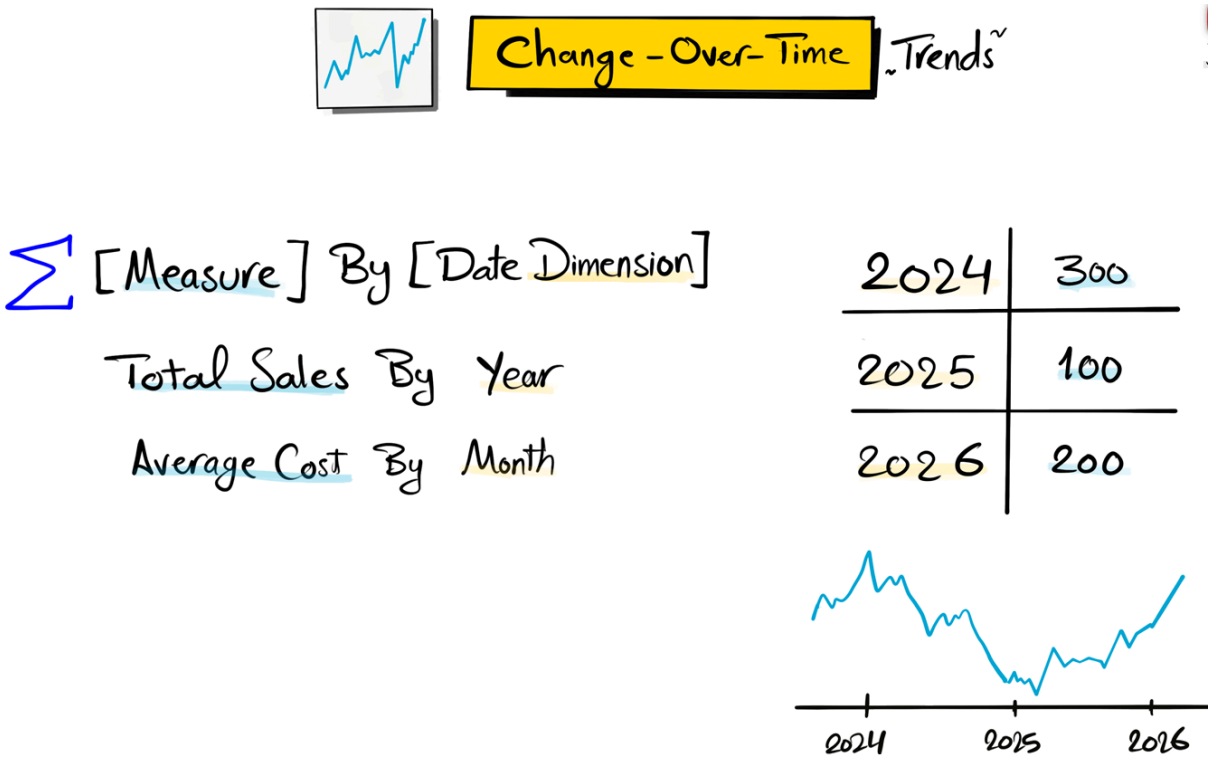
5. Key Analytical Outcomes

- ***Revenue Distribution:*** *Analysis revealed that a specific high-value segment (VIPs) contributes a disproportionate percentage of total revenue despite being a smaller group.*
- ***Product Performance:*** *Cost-range segmentation identified the "Sweet Spot" price points that drive the highest volume versus those that drive the highest margins.*
- ***Growth Velocity:*** *Cumulative sales tracking pinpointed exactly when the business hit major revenue milestones.*

---

6. Technical Planning & Documentation

The project was built following a rigorous "Design-First" approach. Detailed technical sketches and a strategic roadmap were created to map out table relationships and join logic prior to the implementation of the SQL scripts.



Magnitude

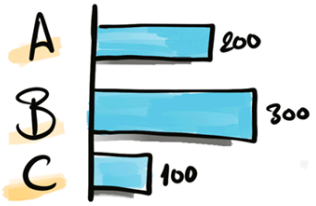
600

Σ [Measure] By [Dimension]

Total Sales By Country

Total Quantity By Category

Average Price By Product

Total Orders By Customer

A	200
B	300
C	100